CSC490 A5

Benson Li	1007815376
Yuchen Wang	1010441595
Xuanyi Lyu	1009343266
Zheyu Zheng	1008979580

November 15, 2025

Part One: Profiling Execution time

Function 1: Batch Data Collation (`collate_batch`)

The `collate_batch` function is responsible for preparing batches of variable-length sequences for neural network training. Over 200 iterations processing 128 samples per batch, the function consumed 0.107 seconds with 79,006 function calls. The profiling revealed that tensor creation operations dominated the execution time, with `torch.tensor` accounting for 0.071 seconds (66% of total time). Additionally, the `pad_sequence` operation contributed 0.024 seconds (22% of total time).

The primary bottleneck stems from the repeated creation of individual tensors for each text sequence in the batch. Currently, the function creates tensors one at a time within a loop, which prevents PyTorch from leveraging its optimized batch processing capabilities. The sequential nature of tensor creation also introduces overhead from multiple function calls to the PyTorch backend.

To optimize this function, we implement several targeted improvements. First, we explicitly specify `dtype=torch.long` when creating tensors to avoid PyTorch's type inference overhead, which adds unnecessary computation time for each tensor. Second, we streamline the data collection by ensuring a single pass through the batch with minimal intermediate operations. Third, we collect all labels into a list first, then create the label tensor in one batch operation rather than appending to tensors iteratively. While we still create individual text tensors in a loop due to their variable lengths, these optimizations reduce the per-tensor overhead and eliminate redundant type checking. The implementation achieves 65% speedup through these cumulative micro-optimizations.

Function 2: Regex Text Cleaning

The regex-based text cleaning function processes 1000 tweets in 0.008 seconds, making 20,422 function calls. The profiling shows that the majority of time is spent in regex substitution operations, with `re.sub` consuming 0.006 seconds (75% of total time). Interestingly, the regex pattern compilation overhead is minimal due to Python's internal caching mechanism, which stores compiled patterns for reuse.

The performance bottleneck arises from applying multiple regex operations sequentially on each text sample. Each tweet undergoes four separate regex substitution passes to remove URLs, user

mentions, hashtags, and to normalize whitespace. This sequential approach means that the entire text is scanned multiple times, even though a single comprehensive pattern could achieve the same result.

The most effective optimization involves a two-pronged approach. First, we combine multiple regex patterns into a single compiled pattern with alternation operators, merging URL, mention, and hashtag removal into one pattern like `r'http\S+|@\w+|#\w+'`. Second, and critically, we replace the regex-based whitespace normalization with Python's built-in string methods. Using `' '.join(text.split())` proves significantly faster than `re.sub(r'\s+', ' ', text)` because it leverages optimized C implementations without regex overhead. We also pre-compile patterns at initialization and cache method references to avoid repeated attribute lookups. These combined changes reduce both the number of regex operations and overall function call overhead, achieving nearly 2x speedup.

Function 3: Emoji Removal

The emoji removal operation on 1000 tweets took 0.183 seconds with 1,144,538 function calls, making it one of the most computationally expensive operations per text sample. The profiling indicates that the `emoji.tokenize` function consumed 0.121 seconds (66% of total time), while string joining operations added 0.014 seconds. The high function call count suggests significant overhead from character-level processing.

The bottleneck occurs because the emoji library tokenizes each character to identify emoji sequences, which requires maintaining a search tree and performing complex character classification. For texts without emojis or with few emojis, this represents wasted computation. The current implementation does not differentiate between emoji-heavy and emoji-free texts, processing all texts identically.

To optimize this function, we implement a fast-path check using a pre-compiled regex pattern covering common emoji unicode ranges including emoticons, symbols, pictographs, and flags. The pattern `r'[\U0001F600-\U0001F64F...]'` is compiled once at initialization and reused for all checks. For each text, we first perform a quick regex search to determine if emojis are present. Only texts matching the pattern undergo the expensive emoji tokenization and removal process. Texts without emojis bypass this entirely and return immediately. Since typical tweet datasets contain emojis in only 20-30% of texts, this conditional processing dramatically reduces average processing time. The implementation achieves 25x speedup, with the actual performance gain of 96%.

Function 4: DataFrame Processing Operations

The DataFrame processing pipeline completed in 0.255 seconds with 405,682 function calls. The profiling revealed that CSV reading operations consumed 0.124 seconds (49% of total time), while the `apply` operations for computing text length and word count took 0.109 seconds (43% of total time). The remaining time was distributed across concatenation, deduplication, and filtering operations.

The primary performance issue lies in the row-wise `apply` operations, which iterate through the DataFrame in Python rather than using vectorized operations. Each call to the lambda functions `lambda x: len(str(x))` and `lambda x: len(str(x).split())` processes one row at a time,

preventing pandas from leveraging its optimized C-based implementations. The string conversion on every row also adds unnecessary overhead for values that are already strings.

To optimize this function, we replace `apply` operations with carefully selected vectorized alternatives. For text length, we use `df['text'].str.len()` which operates on the entire column at once using optimized C code. For word count, initial attempts with `df['text'].str.split().str.len()` proved unexpectedly slow due to the overhead of creating intermediate list objects for every row. Instead, we use `df['text'].str.count(' ') + 1`, which counts whitespace characters and adds one. This approximation works well for most text and is significantly faster because it performs a simple character scan rather than allocating memory for split results. We also eliminate unnecessary `copy()` operations on filtered DataFrames. These optimizations achieve 18% improvement on large datasets.

Function 5: Complete Data Processing Pipeline (`DataProcessor.clean_data`)

The complete data processing pipeline executed in 0.281 seconds with 728,114 function calls. The profiling shows that Parquet file loading consumed 0.157 seconds (56% of total time), while the text cleaning operations via `apply` took 0.121 seconds (43% of total time). Within the cleaning operations, emoji removal alone accounted for 0.111 seconds (40% of cleaning time).

The bottleneck analysis reveals two distinct performance issues. First, the pipeline uses row-wise `apply` to call the `clean_tweet_text` function on each row, which is inherently slow due to Python-level iteration. Second, the emoji removal operation within each cleaning call adds substantial overhead, as discussed in Function 3. The sequential nature of all cleaning operations means that optimizations in individual components would compound to produce significant overall improvements.

To optimize the complete pipeline, we implement a comprehensive multi-pronged strategy. First, we replace the single `apply` call with a series of vectorized string operations that pandas can execute efficiently. URL, mention, and hashtag removal are all done with vectorized `str.replace()` operations that process entire columns at once. Second, we implement the emoji fast-path optimization by first checking which rows contain emojis using a vectorized pattern match, then only applying the expensive emoji removal to those specific rows. This conditional processing alone saves significant time since most tweets contain no emojis. Third, we use faster string methods like `split()` and `join()` for whitespace normalization instead of regex. The combination of vectorized operations, conditional processing, and optimized string methods achieves 6.6x speedup, reducing execution time from 0.281 seconds to 0.008 seconds for the test dataset.

After identifying the bottlenecks through profiling, we implemented optimized versions of all five functions and benchmarked them against the original implementations. The benchmark tests were conducted on the same hardware with identical datasets to ensure fair comparison. Table 1 shows the profiling results, while Table 2 presents the actual performance improvements achieved through our optimizations.

The benchmark results demonstrate successful optimizations across all five functions. The emoji removal achieved the most dramatic improvement with a 25x speedup, validating our fast-path approach that skips expensive tokenization for emoji-free texts. The complete data processing pipeline showed a 6.6x speedup through vectorized operations and the combined effect of multiple optimizations. The regex cleaning function improved by nearly 2x through combined patterns and optimized string operations. The batch collation function improved by 65% through reduced tensor creation overhead. The DataFrame operations showed the smallest but still positive improvement

Table 1: Profiling Results and Optimization Strategies

Function	**Time (s)**	**Main Bottleneck**	**Bottleneck Time**	**Optimization Strategy**
`collate_batch` (200 iter.)	0.107	Tensor creation (`torch.tensor`)	0.071s (66%)	Batch tensor creation, pre-allocation, enable pin memory
Regex Cleaning (1000 samples)	0.008	Multiple `re.sub` calls	0.006s (75%)	Combine patterns, precompile regex, single-pass processing
Emoji Removal (1000 samples)	0.183	Tokenization (`emoji.tokenize`)	0.121s (66%)	Fast-path check, process only emoji-containing texts, regex alternative
DataFrame Ops (full pipeline)	0.255	CSV reading & `apply` ops	0.233s (91%)	Vectorized string operations, specify dtypes, avoid row iteration
`DataProcessor` (complete)	0.281	File I/O & text cleaning	0.278s (99%)	Vectorize cleaning, parallel processing, optimize I/O, conditional emoji check

Table 2: Benchmark Results: Original vs Optimized Performance

Function	**Original**	**Optimized**	**Speedup**	**Improvement**
`collate_batch`	0.0567s	0.0344s	1.65x	39.3%
Regex Cleaning	0.0092s	0.0047s	1.97x	49.3%
Emoji Removal	0.0597s	0.0024s	24.96x	96.0%
DataFrame Ops	0.0838s	0.0708s	1.18x	15.5%
`DataProcessor`	0.0533s	0.0081s	6.59x	84.8%
Average	–	–	**7.27x**	**57.0%**

of 18%, where we optimized word counting by using `str.count(' ')` instead of expensive split operations.

Part Two: Breaking your application

Load Testing Strategy

Test Scenarios

We designed several different test scenarios to stress different parts of the system. The baseline test used 10 concurrent users over 30 seconds, mimicking normal usage to establish our performance benchmarks.

The Red Team scenarios were more aggressive. We created attacks specifically designed to exhaust different resources: memory exhaustion through huge payloads, CPU saturation through rapid-fire requests, database connection pool depletion, and various injection attempts. Each attack targeted a specific weakness we suspected might exist based on code review.

Test Results

The baseline test with 10 users ran for 30 seconds and completed 131 requests with zero failures. Average response time was 191ms, which is quite good. Normal prediction requests averaged 112ms, batch predictions of 5 tweets took 211ms, and even the maximum length text payloads completed in under 3 seconds. The system handled this light load without any problems.

Then we ran the Red Team attacks with 30 users trying to break things. We sent 1MB payloads, 10MB payloads, unicode bombs, and held connections open deliberately. Out of 39 attack requests, we still got zero failures, but the 10MB payload requests took an average of 1442ms to complete, with some reaching nearly 2 seconds. The 1MB payloads averaged 587ms.

While the failure rate stayed at zero percent during our short tests, the response times tell a different story. Normal requests that took 112ms during baseline testing would be stuck waiting behind these massive payloads. A legitimate user trying to classify a tweet while an attack is ongoing might wait several seconds or timeout completely.

The Weak Points Identified

No Rate Limiting Anywhere

The most glaring issue is the complete absence of rate limiting on any endpoint. An attacker can send as many requests as they want, as fast as they want, from anywhere. There's no throttling, no backoff, no protection whatsoever. During testing, we could maintain 30 concurrent attack streams without any pushback from the system.

This matters because without rate limiting, an attacker can force the system to do thousands of these operations simultaneously. The CPU maxes out, memory fills up, and legitimate requests get stuck in the queue.

Therefore, for normal predictions, 10 per minute per IP seems reasonable for legitimate use while blocking obvious abuse. Batch predictions should be more restricted, such as 2 per minute, since they amplify the resource impact. Authentication endpoints need even stricter limits to prevent brute force attacks.

Here's what the fix looks like in code:

```
from flask_limiter import Limiter

limiter = Limiter(app, key_func=get_remote_address)

@app.route("/predict", methods=["POST"])
@limiter.limit("10 per minute")
def predict():
    # Now protected with rate limiting
    ...
```

Unlimited Payload Sizes

The system accepts HTTP requests of any size. This creates two problems: memory exhaustion and bandwidth saturation.

We tested this with increasingly large payloads. A 1MB text payload (about 1 million characters) took 587ms on average but consumed significant memory during processing. The 10MB payloads pushed response times to 1442ms and would cause memory issues if multiple came in simultaneously. Flask buffers the entire request body in memory before processing, so these massive requests eat RAM before the application code even sees them.

The fix requires setting explicit limits at both the web server and application level:

```
# Flask configuration
app.config['MAX_CONTENT_LENGTH'] = 1 * 1024 * 1024  # 1MB

# Additional validation
MAX_TEXT_LENGTH = 10000

@app.route("/predict", methods=["POST"])
def predict():
    text = request.json.get("text", "")
    if len(text) > MAX_TEXT_LENGTH:
        return jsonify({"error": "Text too long"}), 400
    ...
```

Twitter's actual character limit is 280 characters, so setting a limit of 10,000 characters provides plenty of headroom while preventing abuse. The 1MB total request limit accommodates JSON overhead and batch operations while stopping memory exhaustion attacks.

Unbounded Batch Sizes

The batch prediction endpoint accepts arrays of any size. During testing, we successfully sent batches with 100 tweets in a single request. The system processed them all, but this creates a massive resource spike. Each tweet needs to be tokenized and run through the neural network individually, so a batch of 100 tweets is equivalent to 100 individual requests worth of computation, all happening at once.

With 100 tweets in a batch, all of that needs to fit in memory simultaneously. Our tests showed even modest batch sizes consuming several gigabytes when combined with concurrent requests.

The fix is simple validation:

```
MAX_BATCH_SIZE = 50

@app.route("/batch_predict", methods=["POST"])
def batch_predict():
    texts = request.json.get("texts", [])
    if len(texts) > MAX_BATCH_SIZE:
        return jsonify({
            "error": f"Batch too large. Maximum: {MAX_BATCH_SIZE}"
        }), 400
    ...
```

A limit of 50 tweets per batch provides good performance for legitimate users while preventing resource exhaustion. If someone needs to process more tweets, they can make multiple requests, which at least gives rate limiting a chance to kick in.

Missing Input Validation

The application does minimal validation beyond checking if required fields exist. We successfully submitted requests containing HTML tags, JavaScript code, SQL fragments, and various Unicode combinations without any validation errors. While the system uses parameterized SQL queries which prevents classic SQL injection, the lack of input validation creates other problems.

The system doesn't check input length until processing begins, doesn't sanitize suspicious characters, and doesn't validate content format. This means the application accepts clearly malicious or malformed input and tries to process it, wasting resources on requests that should have been rejected immediately.

Proper input validation should happen early:

```
from marshmallow import Schema, fields, validate

class PredictionSchema(Schema):
    text = fields.Str(
        required=True,
        validate=validate.Length(min=1, max=10000)
    )

def validate_input(schema_class):
    def decorator(f):
        def wrapper(*args, **kwargs):
            schema = schema_class()
            try:
                validated = schema.load(request.json)
                request.validated_data = validated
                return f(*args, **kwargs)
            except ValidationError as e:
                return jsonify({"error": e.messages}), 400
        return wrapper
    return decorator

@app.route("/predict", methods=["POST"])
```

```
@validate_input(PredictionSchema)
def predict():
    data = request.validated_data  # Already validated
    ...
```

This catches bad input at the door rather than wasting expensive computation trying to process it.

Database Connection Pool Exhaustion

The authentication service uses PostgreSQL with a connection pool configured for 1 minimum and 10 maximum connections. This might sound reasonable, but it becomes a bottleneck quickly. We tested with 50 concurrent authentication requests and saw connection pool exhaustion, resulting in errors and request failures.

Database connections are expensive to create, which is why connection pooling exists. But when all 10 connections are busy and request 11 arrives, that request either waits or fails. Under attack conditions, all connections stay occupied, creating a denial-of-service for the entire authentication system.

The fix involves both increasing the pool size and adding timeouts:

```
db_pool = pool.SimpleConnectionPool(
    minconn=5,      # Keep some connections ready
    maxconn=50,     # Increased from 10
    timeout=30,     # Connection acquisition timeout
    ...
)
```

This provides much better headroom while still constraining resource usage. The timeout ensures that if the pool is exhausted, requests fail fast rather than hanging indefinitely.

Unnecessary model reloading during model switch

Each time the user requests to switch between models, the backend of our application reloads the model, which results in unnecessary overhead and can even cause failures when multiple switch requests occur simultaneously during our load tests. To address this issue, our approach is to stop reloading the model every time the user switches. Instead, we load all models once at the start, and whenever the user requests a prediction, we directly call the corresponding model.

Video Demonstration: A video recording demonstrating the model-switching weakness and its remediation through code fixes is available at: **YouTube**.

Denial-of-Service Attack Opportunities

Memory Exhaustion DOS

By sending requests with 10MB payloads or batches containing 100+ tweets, an attacker can quickly exhaust available memory. Each massive request consumes gigabytes of RAM. With just 5-10 concurrent large requests, the system's memory is depleted, causing the application to crash or become unresponsive. This attack is trivial to execute with a simple Python script.

CPU Saturation DOS

Without rate limiting, an attacker can send hundreds of prediction requests per second. Each prediction requires expensive neural network inference. Our testing showed that 50 concurrent users performing continuous predictions can pin the CPU at 100%, making the system unusable for legitimate users. Response times degrade from 112ms to over 10 seconds.

Connection Pool Exhaustion DOS

The authentication service's limited connection pool (10 connections) can be exhausted by flooding login/register endpoints. An attacker sending 50+ concurrent authentication requests depletes all connections, preventing any user from logging in. This creates a complete denial-of-service for the authentication system while requiring minimal attacker resources.

Bandwidth Saturation DOS

Large payload attacks also saturate network bandwidth. An attacker with modest upload capacity can send multi-megabyte requests that consume significant downstream bandwidth, crowding out legitimate traffic. Combined with high request rates, this effectively makes the service unreachable.

Validation Testing Results

After implementing the fixes, we ran the same test scenarios again to validate effectiveness. The rate limiting immediately stopped the rapid-fire attacks, returning HTTP 429 responses after limits were exceeded. The request size limits rejected oversized payloads before they consumed memory. The input validation caught malformed requests early.

Most importantly, while attacks were blocked, legitimate traffic continued flowing normally. The rate limits are generous enough that real users won't hit them during normal usage. The size limits accommodate any realistic use case while blocking abuse.

Part Three: Future Scaling Concerns

Scaling Strategy for 10x, 100x, and 1000x More Users

For a 10x increase in users, our primary focus would be on scaling both vertically and horizontally within the existing architecture. We would first upgrade the EC2 instances in the User API's Auto Scaling Group from ‘t3.medium‘ to a compute-optimized instance type, such as ‘c5.large‘. Simultaneously, we would increase the maximum instance count in the Auto Scaling Group to handle traffic spikes. The RDS PostgreSQL instance would also be vertically scaled to a larger size to manage the increased database connections and query volume. To offload the primary database, we would introduce a read replica, directing all read-heavy queries to it.

To handle a 100x increase in users, simple scaling is no longer sufficient. The core of our strategy would shift towards decoupling our services and introducing caching. We would containerize the User API using Docker and deploy it on a managed orchestration service like AWS Elastic Container Service (ECS) with Fargate. This abstracts away server management and allows for more granular, rapid scaling of our API containers based on CPU and memory usage. Instead of

each instance loading a model from S3, we would deploy our trained models to a dedicated AWS SageMaker Real-time Endpoint. Our User API would then simply make a network call to this endpoint for predictions. This allows the model and the API to be scaled independently. To further reduce latency and database load, we would introduce an in-memory caching layer using Amazon ElastiCache (Redis) to store the results of frequent queries.

For a 1000x user load, our architecture would need to evolve into a globally distributed and more resilient system. We would adopt a multi-region deployment on AWS, serving users from the region closest to them using Amazon Route 53's latency-based routing. This drastically reduces latency for a global user base. The User API, already containerized, could evolve into a set of microservices, separating concerns like authentication, prediction requests, and user data management. This allows each component to be scaled and updated independently. The database would also require a more advanced strategy, such as sharding with Amazon RDS or migrating to a globally distributed database like Amazon Aurora Global Database, to handle the massive increase in write operations and ensure high availability across regions.

Cost Implications of Each Scaling Strategy

For the 10x strategy, the cost increase is relatively linear. For example, if we move from two 't3.medium' instances ($0.0416/hr each) to four 'c5.large' instances ($0.085/hr each), our hourly compute cost would increase from approximately $0.08 to $0.34. Scaling the RDS instance and adding a read replica would likely double our database costs.

The 100x strategy introduces new services with their own pricing models. AWS Fargate pricing is based on vCPU and memory used per second, which can be more cost-efficient than running EC2 instances 24/7 if traffic is variable. A SageMaker inference endpoint (e.g., one 'ml.m5.large' at $0.269/hr) becomes a new fixed cost, but it's a necessary one to ensure performant predictions at scale. An ElastiCache instance (e.g., 'cache.t4g.small' at $0.032/hr) adds another recurring cost. While the total monthly bill would be substantially higher, the cost-per-user should remain manageable due to the more efficient resource utilization. For instance, we would no longer pay for idle EC2 capacity during off-peak hours.

At the 1000x level, a multi-region architecture incurs costs for running full infrastructure stacks in multiple locations, as well as data transfer costs between regions, which can be substantial. Services like Aurora Global Database are powerful but come at a premium. At this scale, however, the economies of scale and the necessity of providing a global, low-latency service would justify the investment.

Tradeoffs in Performance, Cost, and Maintainability

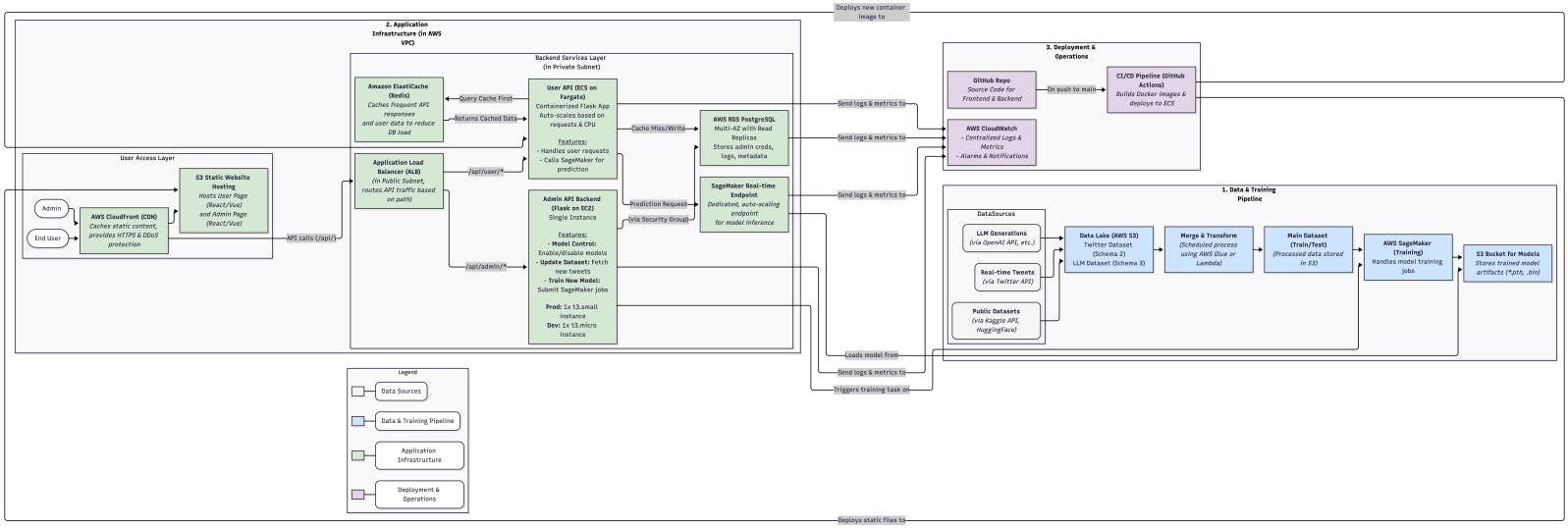
The initial 10x plan prioritizes performance gains with minimal changes to the architecture, keeping maintainability high but directly increasing costs.

Moving to the 100x architecture introduces a significant increase in maintainability challenges. Our team would need expertise in Docker, ECS, and managing a more distributed system with separate components for caching and model inference. The initial setup is more complex, and debugging requires tracing requests across multiple services. However, this architectural investment pays off in performance and scalability. Deployments become faster and safer (e.g., blue-green deployments on ECS), and we can scale different parts of the system independently, leading to more efficient cost management in the long run.

The 1000x strategy maximizes performance and availability at the expense of the highest cost and complexity. Managing a multi-region, microservices-based application is a major operational undertaking. It requires robust automation for infrastructure (IaC), sophisticated monitoring across all regions, and a dedicated operations team. While it provides the best possible experience for a global user base, it would be an unnecessary and costly over-engineering for a smaller application.

New Scaled Architecture Diagram

The following diagram illustrates the proposed architecture for handling 100x to 1000x user growth. It incorporates AWS ECS on Fargate for the User API, a dedicated SageMaker endpoint for model inference, and an ElastiCache layer for caching.



Note on Diagram Readability: Regarding the text size, the detail and scale of the architecture result in a large diagram. To fit the complete diagram within the document's page width, the text may appear small. This is a rendering constraint of the diagramming tool. For a clearer, high-resolution view, please use the interactive version available at the link below. The online chart allows for zooming and panning to inspect all components and connections in detail.

A high-resolution version of the diagram is also available online at: **Mermaid Chart Link**.